

WhatsApp Chat Analyzer

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

NANDANI NEEMA-EN23CS301655

NANDINI RATHI-EN23CS301657

Under the Guidance of

Prof. Rashmi Choudhary

Prof. Sonal Modh Bhardwaj



Department of Computer Science & Engineering

Faculty of Engineering

MEDI-CAPS UNIVERSITY, INDORE- 453331

APRIL-2026

WhatsApp Chat Analyzer

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

NANDANI NEEMA-EN23CS301655

NANDINI RATHI-EN23CS301657

Under the Guidance of

Prof. Rashmi Choudhary

Prof. Sonal Modh Bhardwaj



Department of Computer Science & Engineering

Faculty of Engineering

MEDI-CAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work“**WhatsApp Chat Analyzer**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled “**WhatsApp Chat Analyzer**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science and Engineering’ completed under the supervision of <**Name, designation and department of the Guide(s)**>, Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

I/We, <Name of the Guide(s)> certify that the project entitled “**WhatsApp Chat Analyzer**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Nandani Neema and Nandini Rathi** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

<Name of Internal Guide>

<Name of the Department>

Medi-Caps University, Indore

<Name of External Guide (If any)>

<Name of the Department>

Name of the Organization

Dr. Ratnesh Litoriya

Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Pramod S. Nair**, Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Ratnesh Litoriya** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Nandani Neema

Nandini Rathi

B.Tech. III Year (K)

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

A WhatsApp Chat Analyzer is a data-driven project designed to extract meaningful insights from chat conversations using data analysis and visualization techniques. With the rapid growth of digital communication, messaging platforms like WhatsApp generate large volumes of textual data daily. This project aims to analyze such data to uncover patterns, trends, and user behavior in conversations.

The analyzer processes exported WhatsApp chat data and performs various preprocessing steps, including data cleaning, formatting, and structuring. It identifies key attributes such as message frequency, active users, word usage, emojis, and time-based activity. Using tools like Python along with libraries such as Pandas, Matplotlib, and Seaborn, the system generates visual representations including graphs and charts for better understanding.

The project provides insights such as the most active participant, busiest time periods, most commonly used words, and emoji analysis. It also highlights message distribution over days, months, and hours, helping users understand communication patterns. Advanced features may include sentiment analysis to determine the emotional tone of messages and word cloud generation to visualize frequently used terms.

The WhatsApp Chat Analyzer is useful for both personal and professional purposes. Individuals can explore their chat habits, while organizations can analyze communication trends in team conversations. The project demonstrates the practical application of data analytics, natural language processing, and visualization in real-world scenarios.

In conclusion, this project showcases how unstructured textual data from messaging platforms can be transformed into valuable insights. It enhances decision-making, improves understanding of communication behavior, and highlights the importance of data analysis in modern digital interactions.

Keywords:

WhatsApp Chat Analyzer, Data Analysis, Data Visualization, Python, Pandas, Matplotlib, Seaborn, Natural Language Processing (NLP), Word Cloud , Emoji Analysis.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
Chapter 1	INTRODUCTION	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	3-5
	1.4 Significance	5
	1.5 Research Design	6
	1.6 Source of Data	6
	1.7 Chapter Scheme	7
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteristics	8
	2.2 Functional Requirements	8-9
	2.3 Dependencies	10
	2.4 Performance Requirements	10-11
	2.5 Hardware Requirements	11-12
	2.6 Constraints & Assumptions	12
Chapter 3	DESIGN	
	3.1 Algorithm	13
	3.2 System Design	14-15
	3.3.1 Data Flow Diagram	15
	3.3.2 Activity Diagram	15
	3.3.3 Flow Chart	16
	3.3.4 Sequence diagram	17
Chapter 4	Implementation, Testing, and Maintenance	

	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	18
	4.2 Testing Techniques and Test Plans	18-19
	4.3 Installation Instructions	19
	4.4 End User Instructions	19
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	20
	5.2 Brief Description of Various Modules of the system	21
	5.3 Snapshots of system with brief detail of each	22-25
Chapter 6	Summary and Conclusions	26
Chapter 7	Future scope	27
	Appendix	28
	Bibliography	28

List of Figures

Figure No.	Title	Page No.
1.1	Introduction to WhatsApp Chat Analyzer	1
3.2.1	Dataflow Diagram	14
3.2.2	Activity Diagram	15
3.2.3	Flowchart	16
3.2.4	Sequence Diagram	17
5.3.1	Chat file upload and dataframe creation	22
5.3.2	Monthly timeline graph visualization	23
5.3.3	Daily timeline graph visualization	23
5.3.4	Activity map	24
5.3.5	Word cloud visualization	25
5.4.6	Emoji analysis	25

List of Tables

Table No.	Title	Page No.
2.1	Functional Requirements of System	8
2.2	User Characteristics	9
2.3	Hardware and Software Requirements	12
4.1	Tool and technologies used	18

Abbreviations

Abbreviation	Full Form
NLP	Natural Language Processing
UI	User Interface
IDE	Integrated Development Environment
DFD	Data Flow Diagram
CSV	Comma Separated Values
API	Application Programming Interface

Chapter-1

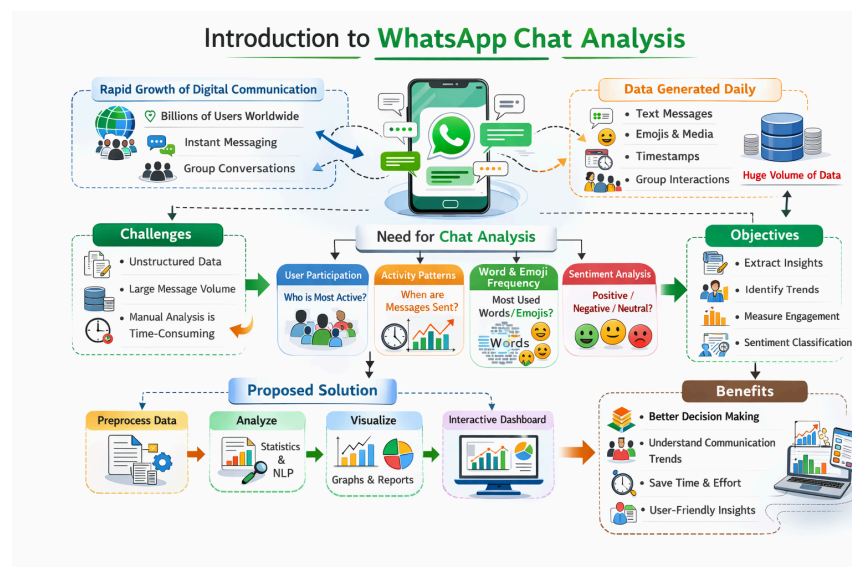
Introduction

1.1 Introduction :

In the modern era of digital communication, instant messaging platforms have become the backbone of interpersonal and professional interaction. Among these platforms, WhatsApp stands as one of the most widely used messaging applications globally. With billions of active users, WhatsApp facilitates the exchange of text messages, multimedia files, voice notes, emojis, hyperlinks, and documents on a daily basis. The enormous amount of conversational data generated creates an opportunity for analytical exploration.

Despite its popularity, WhatsApp does not provide analytical tools to evaluate chat activity, communication patterns, user engagement levels, or emotional tone of conversations. Although users can export chats in text format, the exported file remains unstructured and difficult to interpret manually. Identifying patterns in long conversations, especially in group chats, requires automated processing and analytical tools.

The WhatsApp Chat Analyser project is designed to address this gap by transforming exported chat text into structured, meaningful insights using Data Science and Natural Language Processing techniques. The system preprocesses raw text data, extracts important features, performs statistical analysis, and generates visual representations to make communication insights easily understandable. This project demonstrates the practical application of Python programming, data preprocessing, text mining, exploratory data analysis, and web deployment using modern frameworks. It serves as a real-world implementation of analytics concepts in social communication systems.



1.2 Literature Review :

Text mining and Natural Language Processing (NLP) are widely used techniques for analyzing unstructured textual data. Many researchers have worked on extracting meaningful information from social media platforms and online communication systems.

Pang and Lee (2008) conducted foundational research in opinion mining and sentiment analysis, where they discussed computational approaches for classifying text into positive, negative, and neutral categories. Their work established important techniques for analyzing textual sentiment, which are applicable to chat-based communication systems.

Jurafsky and Martin (2019), in their book *Speech and Language Processing*, provided detailed explanations of NLP techniques such as tokenization, text preprocessing, feature extraction, and sentiment classification. These methods are commonly applied when analyzing conversational data.

Aggarwal (2011), in *Social Network Data Analytics*, discussed how communication data from social platforms can be analyzed to understand user behavior, participation patterns, and interaction trends. His work emphasizes the importance of extracting structured information from unstructured communication logs.

Feldman and Sanger (2007) explained various text mining techniques used to process large volumes of unstructured textual data. They highlighted preprocessing, pattern discovery, and statistical analysis as essential steps in textual data analysis.

While extensive research exists on social media analytics and sentiment analysis, limited academic work specifically focuses on automated analysis of WhatsApp exported chat data with integrated visualization dashboards. Most prior studies analyze public platforms such as Twitter and Facebook rather than private messaging data.

Therefore, the proposed WhatsApp Chat Analyser project builds upon established research in:

- Text preprocessing techniques
- Sentiment analysis methodologies
- Social network behavior analysis
- Data visualization methods

and applies these techniques specifically to exported WhatsApp chat datasets.

1.3 Objectives :

1. To Read and Preprocess Exported WhatsApp Chat Files

The first objective of the WhatsApp Chat Analyser is to read exported chat files in `.txt` format and preprocess them for analysis. The exported chat file typically contains timestamps, sender names, system-generated messages, and message content in a continuous unstructured format.

Preprocessing involves:

- Importing the text file into the system
- Removing unnecessary system messages (e.g., “User joined the group”)
- Handling media placeholders such as “<Media omitted>”
- Separating date, time, sender name, and message body
- Converting date and time into a standardized datetime format

This step ensures that raw chat data is cleaned and structured properly before further analysis. Proper preprocessing improves accuracy and reliability of results.

2. To Convert Unstructured Chat Text into Structured Tabular Data

After preprocessing, the next objective is to transform the cleaned textual data into a structured format such as a DataFrame (table structure). Structured data enables efficient analysis and computation.

The system organizes data into columns such as:

- Date
- Time
- User
- Message
- Word Count
- Sentiment (if applied)

Converting unstructured text into tabular format allows for grouping, filtering, sorting, and applying statistical operations easily. This transformation is essential for performing advanced analytical operations.

3. To Calculate Overall Chat Statistics such as Total Messages and Word Count

Another key objective is to generate overall statistical insights about the conversation. These statistics provide a quick summary of chat activity.

The system calculates:

- Total number of messages exchanged

- Total number of words used
 - Total media files shared
 - Total links shared
 - Average messages per day
-

4. To Identify User-wise Participation in Group Chats

In group chats, participation levels vary among members. This objective focuses on analyzing individual user contributions.

The system:

- Counts the total messages sent by each participant
- Calculates percentage contribution of each user
- Ranks users based on activity level

This helps identify the most active and least active members in the group. Such analysis can be useful for understanding communication dominance and engagement levels.

5. To Analyze Daily, Monthly, and Yearly Communication Trends

Time-based analysis is crucial to observe how communication evolves over time. This objective aims to detect patterns in message frequency across different time intervals.

The system performs:

- Daily message count analysis
- Monthly activity trends
- Yearly growth or decline in communication
- Hourly activity distribution

This allows users to identify peak chatting hours, busiest days, and long-term communication changes using time-series analysis.

6. To Perform Emoji and Word Frequency Analysis

Textual communication often includes commonly repeated words and emojis that reflect communication style and emotional expression.

For word frequency analysis, the system:

- Tokenizes text into individual words
- Removes stop words (e.g., “the”, “is”, “and”)
- Counts frequency of remaining meaningful words

For emoji analysis, the system:

- Extracts emojis from messages
- Counts their occurrences
- Identifies most frequently used emojis

This objective helps reveal conversation themes and emotional tendencies.

8. To Design an Interactive Web Interface for User-Friendly Experience

The final objective is to ensure usability by developing an interactive web interface using frameworks such as Streamlit.

The interface allows users to:

- Upload exported chat files
- Select specific users for analysis
- View graphical visualizations
- Download reports (if implemented)

A user-friendly interface ensures that even non-technical users can easily interact with the system and interpret the results without needing programming knowledge.

1.4 Significance :

The WhatsApp Chat Analyzer project holds significant value in the field of data analytics as it demonstrates how unstructured textual data can be transformed into meaningful insights. With the increasing use of messaging platforms for both personal and professional communication, large amounts of data are generated daily. This project highlights the importance of analyzing such data to understand communication patterns, user behavior, and trends.

The project is significant as it applies core concepts of data analysis and data visualization in a practical scenario. By using tools like Python, Pandas, Matplotlib, and Seaborn, it showcases how analytical techniques can be used to extract useful information such as message frequency, active users, commonly used words, and emoji usage. These insights help in better understanding interaction habits and communication dynamics.

From a practical perspective, the analyzer can be useful for individuals to evaluate their chat usage patterns and for organizations to analyze team communication efficiency. It can help identify peak activity periods, engagement levels, and behavioral trends within conversations. This can further support decision-making and improve communication strategies.

Additionally, the project enhances technical skills such as data preprocessing, visualization, and handling real-world datasets. It also serves as a foundation for more advanced applications like chatbot development and predictive analytics.

In conclusion, the WhatsApp Chat Analyzer project is significant as it bridges theoretical knowledge with real-world application, demonstrating the growing importance of data analytics in understanding digital communication.

1.5 Research Design :

The research design for the WhatsApp Chat Analyzer project is based on a data-driven and experimental approach, focusing on extracting insights from real-world chat data. The project follows a structured process that includes data collection, preprocessing, analysis, and visualization.

The first step involves data collection, where WhatsApp chat data is exported in text format. This data serves as the primary source for analysis. The collected dataset contains timestamps, user names, and message content, which are essential for understanding communication patterns.

The next phase is data preprocessing, where the raw chat data is cleaned and structured. This includes removing unnecessary information such as system messages, handling missing values, and converting timestamps into a usable format. The data is then organized into a structured form using tools like Python and libraries such as Pandas.

After preprocessing, the project moves to the data analysis phase. Various analytical techniques are applied to extract meaningful insights, such as calculating message frequency, identifying active users, analyzing word usage, and examining time-based activity patterns. Statistical and exploratory data analysis methods are used to interpret the data effectively.

The visualization phase follows, where the analyzed data is represented through graphs and charts using libraries like Matplotlib and Seaborn. Visualizations such as timelines, bar charts, and word clouds help in presenting the results in an easy-to-understand manner.

Finally, the results are evaluated to understand communication behavior and patterns. The overall research design ensures a systematic approach to transforming raw chat data into useful insights, making the project practical and relevant in the field of data analytics.

1.6 Source of Data :

The primary source of data for the WhatsApp Chat Analyzer project is exported chat data from WhatsApp conversations. WhatsApp provides a built-in feature that allows users to export their chat history in a text (.txt) format. This exported file contains essential information such as message timestamps, sender names, and message content, which are used for analysis.

The data used in this project is secondary data, as it is already generated through real-world communication and then collected for analytical purposes. The dataset may include individual or group chats depending on the scope of the project. Only textual data is considered, while media files such as images, videos, and audio are excluded from analysis.

To obtain the dataset, the chat is exported using the “Export Chat” feature available in WhatsApp settings. The exported file is then processed and used as input for the analyzer. Before analysis, necessary preprocessing steps such as data cleaning, formatting, and structuring are performed to ensure accuracy and consistency.

The data used in this project is limited to sample or personal chat data and is handled with care to maintain privacy and confidentiality. No sensitive or personal information is disclosed during the analysis or presentation of results.

1.7 Chapter Scheme :

The project report is organized into different chapters to ensure a systematic and clear presentation of the work carried out in the WhatsApp Chat Analyzer project.

Chapter 1: Introduction

This chapter provides an overview of the project, including the background, objectives, significance, research design, and source of data. It sets the context for the study and explains the purpose of developing the WhatsApp Chat Analyzer.

Chapter 2: Requirement Specification

This chapter describes the system requirements, including user characteristics, functional requirements, dependencies, performance requirements, hardware requirements, and constraints. It defines what the system is expected to do and the conditions under which it operates.

Chapter 3: System Design

This chapter explains the design of the system. It includes the algorithm used, system architecture, and various diagrams such as data flow diagrams, activity diagrams, and flowcharts. It provides a blueprint for how the system is structured and functions.

Chapter 4: Implementation, Testing, and Maintenance

This chapter discusses the tools and technologies used for implementation, such as Python and relevant libraries. It also covers testing techniques, test cases, installation procedures, and user instructions for operating the system.

Chapter 5: Results and Discussion

This chapter presents the output of the system, including visualizations, user interface representations, and analysis results. It also provides a discussion of the findings and interpretation of the results.

Final Sections

The report concludes with a summary and conclusion of the project, followed by future scope, bibliography, and appendix (if applicable), which may include additional data or code snippets.

This structured organization ensures clarity, logical flow, and ease of understanding for the reader.

Chapter-2

Requirements Specification

2.1 User Characteristics :

The WhatsApp Chat Analyzer is designed for users with basic knowledge of computers and an interest in analyzing chat data. The system is simple and user-friendly, making it accessible to a wide range of users without requiring advanced technical expertise.

General Users:

These include individuals who want to analyze their personal or group chats. They are expected to have basic computer skills such as file handling, uploading data, and interpreting simple graphs and charts. No prior knowledge of programming is required for using the system.

Students and Learners:

Students, especially those in fields like computer science, data analytics, or information technology, can use the analyzer to understand data analysis concepts. They may have a basic understanding of tools like Python and data visualization libraries, but it is not mandatory for basic usage.

Technical Users (Developers/Analysts):

These users may have knowledge of programming, particularly Python, and can modify or extend the project. They are familiar with data preprocessing, analysis techniques, and visualization tools such as Pandas, Matplotlib, and Seaborn.

User type	description	Skills required
General user	individuals who want to analyze personal or group chats	Basic computer knowledge, file handling
student	Learners using the system for academic or learning purposes	Basic understanding of data and graphs
Technical user	Developers or analysts who may modify or extend the system	Knowledge of Python and data analysis tools

2.2 Functional Requirements :

The functional requirements define the core operations and features that the WhatsApp Chat Analyzer system must perform. These requirements describe how the system processes input data and generates meaningful outputs.

1. Chat Data Input:

The system should allow users to upload or provide exported WhatsApp chat data in text (.txt) format for analysis.

2. Data Preprocessing:

The system must clean and structure the raw chat data by removing unnecessary information such as system messages and formatting the data into a usable structure (date, time, user, message).

3. Message Statistics Generation:

The system should calculate and display key statistics such as total number of messages, number of words, number of media messages, and number of links shared.

4. User Activity Analysis:

The system should identify and display the most active users in the chat based on message count and participation.

5. Time-Based Analysis:

The system must analyze chat activity over time, including daily, monthly, and hourly message trends.

6. Word Frequency Analysis:

The system should identify and display the most frequently used words in the chat.

7. Emoji Analysis:

The system should detect and analyze emojis used in the chat and present their frequency.

8. Data Visualization:

The system must generate graphical representations such as bar charts, line graphs, and word clouds for better understanding of the data.

9. Result Display:

The system should present the analyzed results in a clear and user-friendly format.

10. Error Handling:

The system must handle invalid or improperly formatted input files and provide appropriate error messages.

Requirement ID	Description
FR1	Upload WhatsApp chat file
FR2	Preprocess chat data
FR3	Generate message statistics
FR4	Perform user activity analysis

FR5	Generate visualization
-----	------------------------

These functional requirements ensure that the system effectively processes chat data and delivers meaningful insights to the user

2.3 Dependencies :

The WhatsApp Chat Analyzer system depends on various software tools, libraries, and system configurations for its proper functioning. These dependencies are essential for data processing, analysis, and visualization.

1. Programming Language:

The system is developed using **Python**, which serves as the core programming language for implementing data analysis and processing tasks.

2. Libraries and Frameworks:

The project relies on several Python libraries, including:

- **Pandas** – for data manipulation and preprocessing
- **Matplotlib** – for generating graphs and visualizations
- **Seaborn** – for advanced data visualization
- **NumPy** – for numerical operations
- **WordCloud** – for generating word cloud representations
- **Regex (re module)** – for parsing and extracting information from chat data

3. Development Environment (IDE):

The project requires an IDE such as **Jupyter Notebook**, **VS Code**, or **PyCharm** for writing and executing the code.

4. Operating System:

The system can run on common operating systems such as Windows, macOS, or Linux.

5. Input Dependency:

The system depends on properly exported WhatsApp chat data in **.txt format**. Incorrect or unsupported file formats may affect the functioning of the system.

6. Optional Dependencies:

- **Streamlit** – for creating a user interface
- **Web browser** – for viewing the interface

These dependencies ensure that the system runs smoothly and performs accurate data analysis and visualization.

2.4 Performance Requirements :

The performance requirements of the WhatsApp Chat Analyzer define how efficiently and reliably the system should operate while processing and analyzing chat data.

1. Response Time:

The system should process and analyze the uploaded chat file within a reasonable time. For small to medium-sized chat files, the results should be generated within a few seconds. For larger datasets, processing time may increase but should remain acceptable.

2. Scalability:

The system should be capable of handling different sizes of chat data, ranging from small individual chats to large group conversations with thousands of messages, without significant degradation in performance.

3. Accuracy:

The system must provide accurate results in terms of message counts, user activity, word frequency, and other analytical outputs. Proper data preprocessing should ensure reliable results.

4. Efficiency:

The system should use system resources such as memory and CPU efficiently. It should avoid unnecessary computations and optimize data handling using efficient libraries like Pandas and NumPy.

5. Reliability:

The system should function consistently without crashes or errors during normal operation. It should handle unexpected inputs gracefully.

6. Usability Performance:

The interface (if any) should be responsive and easy to use, allowing users to quickly upload data and view results without delays.

7. Data Handling Capability:

The system should efficiently handle large text files and process multiple types of chat content such as messages, emojis, and links.

These performance requirements ensure that the WhatsApp Chat Analyzer delivers fast, accurate, and reliable results for different types of users and datasets

2.5 Hardware Requirements :

The WhatsApp Chat Analyzer does not require high-end hardware and can run efficiently on standard computing systems.

Minimum Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent
- **RAM:** 4 GB
- **Storage:** At least 500 MB free space

- **System Type:** 32-bit or 64-bit computer
- **Display:** Standard monitor for viewing graphs and outputs

Recommended Hardware Requirements:

- **Processor:** Intel Core i5 or higher
- **RAM:** 8 GB or more
- **Storage:** 1 GB or more free space
- **Internet:** Optional (required only if using online tools or libraries)

These specifications ensure smooth execution and faster processing of large chat datasets.

Component	Minimum requirement	Recommended requirement
Processor	Intel core i3	Intel Core i5 or higher
RAM	4GB	8 GB or more
Storage	500MB free space	1GB or above
Display	Standard monitor	High- resolution display

2.6 Constraints and Assumptions :

Constraints:

- The system only supports **WhatsApp exported chat files in .txt format**
- It does not analyze **media files** such as images, videos, or audio
- The accuracy of results depends on the **format and quality of input data**
- The system may experience slower performance with **very large chat files**
- Limited to **text-based analysis** and visualization

Assumptions:

- Users will provide **properly exported and correctly formatted chat data**
- The chat data contains **consistent timestamps and user names**
- Users have **basic knowledge of operating a computer system**
- Required software (Python and libraries) is **properly installed and configured**
- The system will be used in a **controlled environment without corrupted files**

These constraints and assumptions define the boundaries within which the system operations.

Chapter-3

Design

3.1 Algorithm :

The algorithm for the WhatsApp Chat Analyzer defines the step-by-step procedure used to process and analyze chat data.

Step 1: Start

Begin the execution of the system.

Step 2: Input Chat Data

Load the exported WhatsApp chat file in (.txt) format.

Step 3: Data Preprocessing

- Read the text file
- Remove unnecessary system messages (e.g., “user joined”, “media omitted”)
- Extract date, time, user name, and message
- Convert raw data into a structured format (table/dataframe)

Step 4: Data Cleaning

- Handle missing values
- Remove null or invalid entries
- Standardize date and time format

Step 5: Feature Extraction

- Calculate total number of messages
- Count total words
- Identify media messages and links
- Extract emojis from messages

Step 6: User Analysis

- Count number of messages per user
- Identify most active users

Step 7: Time-Based Analysis

- Analyze messages by day, month, and year
- Identify peak activity hours

Step 8: Word Frequency Analysis

- Remove stop words (common words like “the”, “is”)
- Count frequency of remaining words
- Identify most frequently used words

Step 9: Emoji Analysis

- Count frequency of emojis used
- Identify most commonly used emojis

Step 10: Data Visualization

- Generate charts (bar graphs, line graphs)
- Create word cloud for frequent words

Step 11: Display Results

Show all calculated statistics and visualizations to the user.

Step 12: End

Terminate the program.

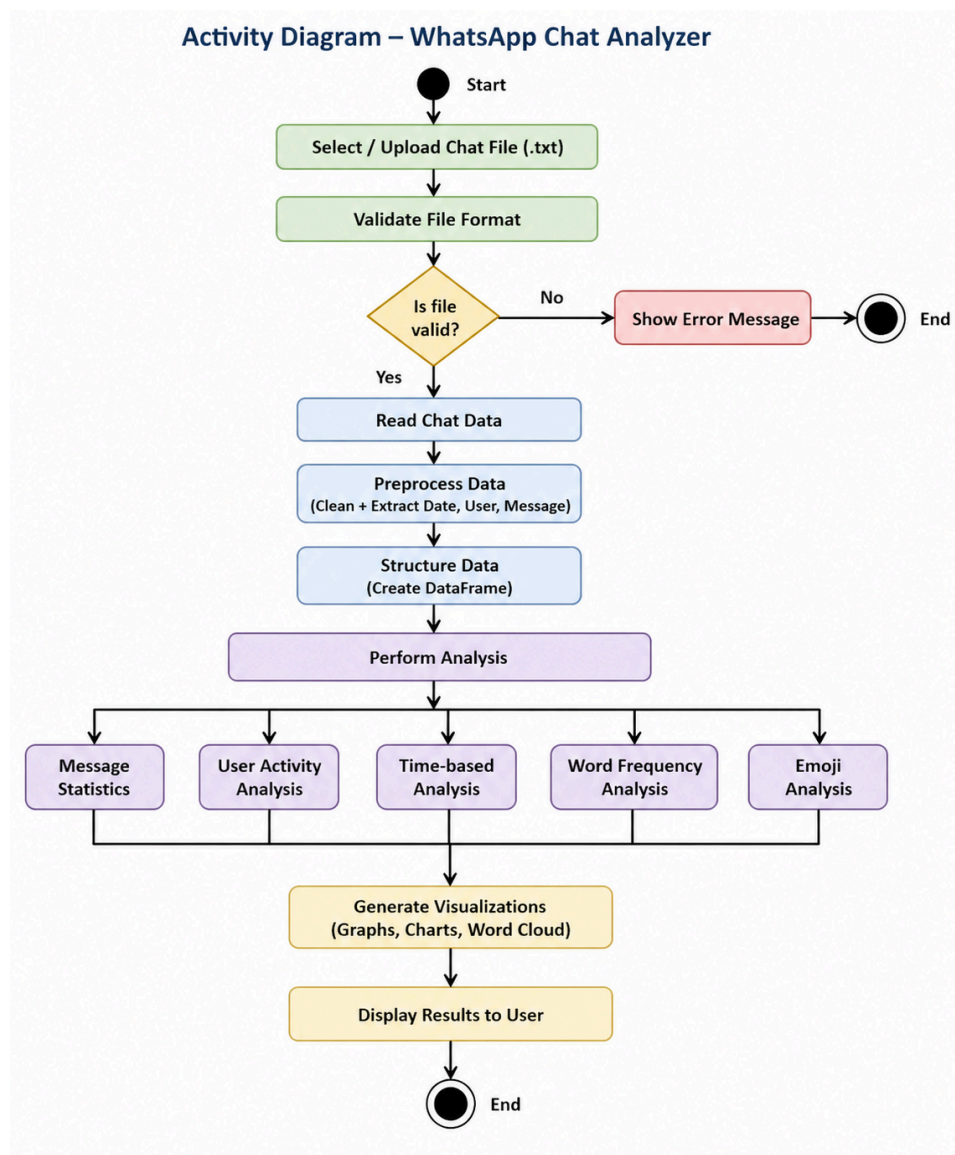
Data Stores

- **D1: Chat Files** – Stores uploaded chat files.
- **D2: Messages** – Stores extracted messages from the chat file.
- **D3: Word & Emoji Data** – Stores analyzed word frequency and emoji usage.
- **D4: Reports** – Stores generated analysis reports.

Summary

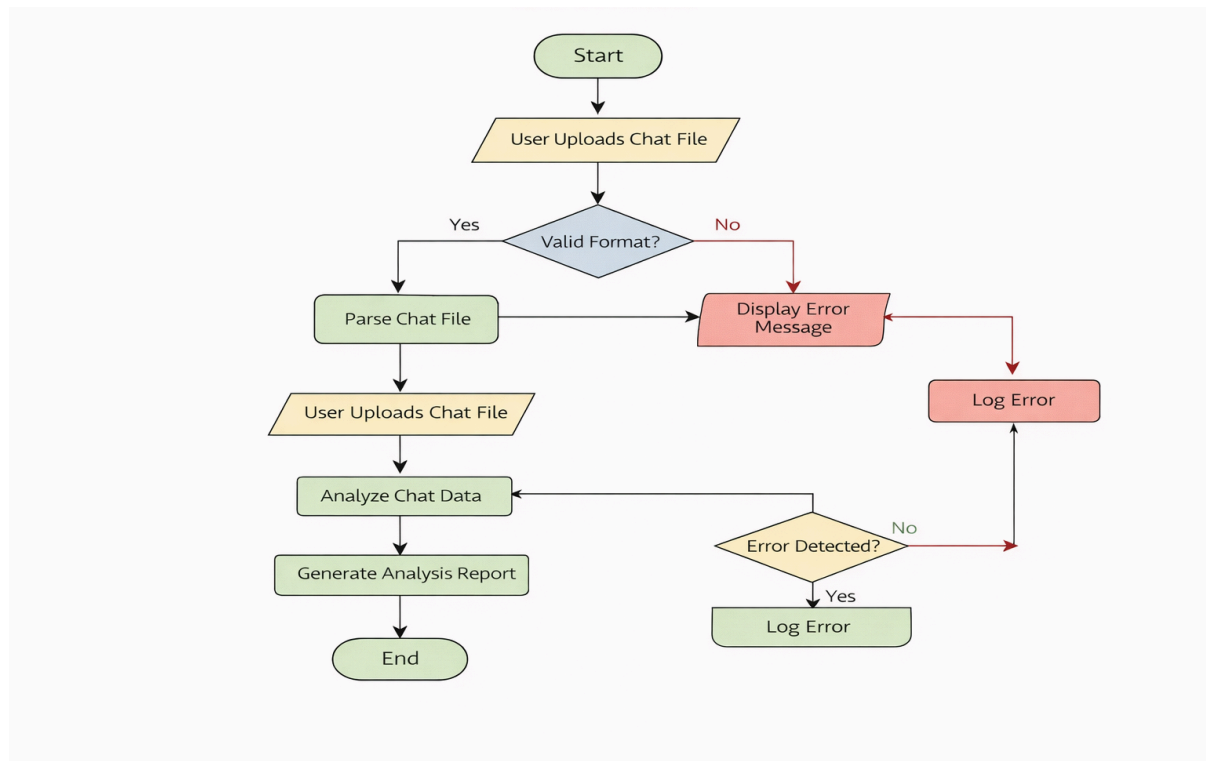
The DFD explains how the system receives chat data, processes it through different stages, stores the results, and generates reports for users and administrators in the WhatsApp Chat Analyzer system.

3.2.2 Activity Diagram :



The activity diagram represents the workflow of the WhatsApp Chat Analyzer system, starting from uploading the chat file to preprocessing, analyzing, generating visualizations, and displaying results. It also includes a validation step to handle incorrect input files.

3.2.3 Flowchart :



The flowchart of the WhatsApp Chat Analyzer represents the step-by-step working process of the system, starting from input to the final output.

The process begins with the Start node, where the system is initiated. The user is then required to select or upload the WhatsApp chat file in (.txt) format. After uploading, the system performs a file format validation step to ensure that the input file is correct and supported.

A decision point follows, where the system checks whether the file is valid or not. If the file is invalid, the system displays an error message and the process terminates. If the file is valid, the system proceeds to the next step.

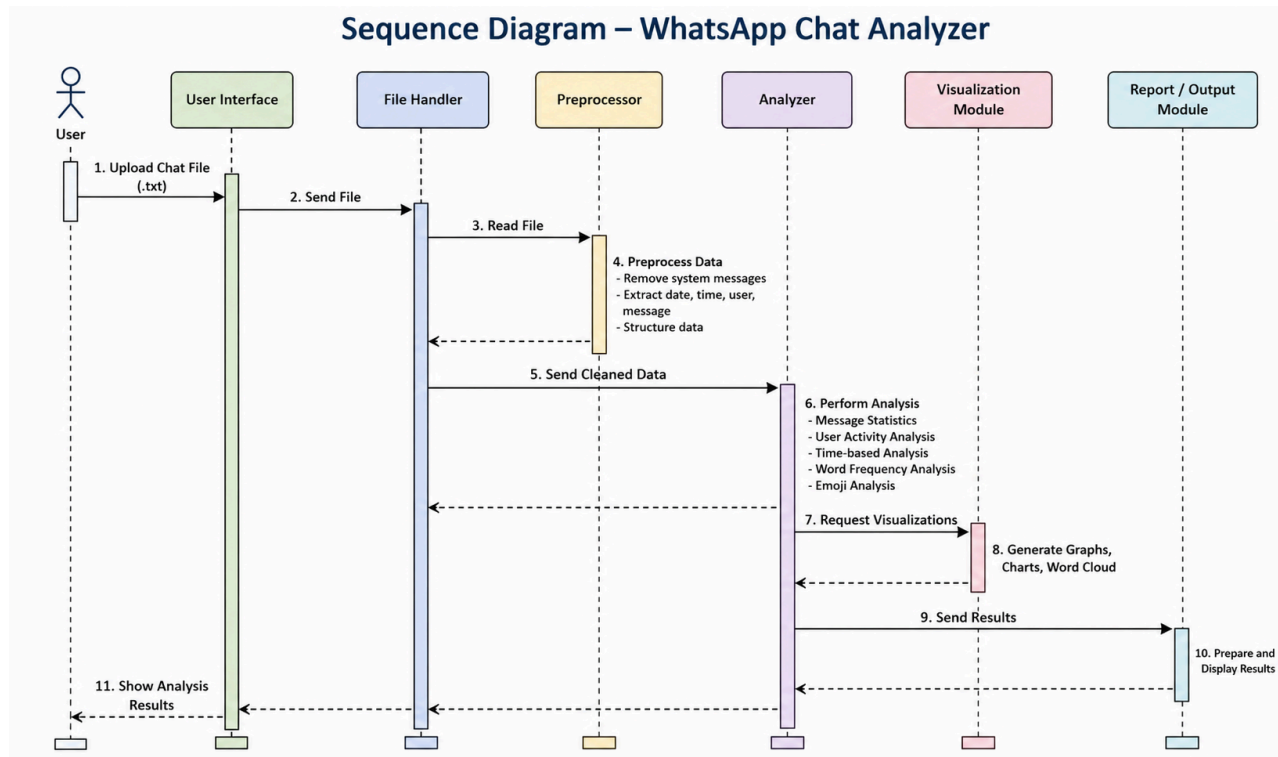
The system then reads the chat data from the file and performs data preprocessing, which includes cleaning the data and extracting important fields such as date, time, user name, and message. The processed data is then structured into a proper format (such as a table or dataframe) for further analysis.

Next, the system performs multiple analyses, including message statistics, user activity analysis, time-based analysis, word frequency analysis, and emoji analysis. These analyses help in identifying patterns and trends in the chat data.

After analysis, the system generates visualizations such as graphs, charts, and word clouds to represent the data in an understandable format. Finally, the system displays the results to the user.

The process ends with the End node, completing the flow of the system.

3.2.4 Sequence Diagram :



The sequence diagram of the WhatsApp Chat Analyzer illustrates the interaction between different components of the system over time. It shows how the user request flows through various modules and how data is processed step by step.

The process begins when the User uploads the WhatsApp chat file through the User Interface. The interface then forwards the file to the File Handler, which is responsible for managing and reading the input file.

The File Handler sends the file to the Preprocessor, where the raw chat data is cleaned and structured. During preprocessing, unnecessary system messages are removed, and important fields such as date, time, user name, and message content are extracted. The cleaned and structured data is then sent back to the system.

Next, the processed data is passed to the Analyzer module. The analyzer performs multiple operations such as calculating message statistics, identifying active users, analyzing time-based activity, computing word frequency, and extracting emoji usage.

After completing the analysis, the Analyzer sends a request to the Visualization Module to generate graphical outputs. The visualization module creates charts, graphs, and word clouds to represent the analyzed data in an understandable format.

The generated results are then forwarded to the Report/Output Module, which prepares the final output. This module organizes the results and ensures they are presented clearly.

Finally, the results are sent back to the User Interface, where they are displayed to the user. The process ends after the user views the analysis results

Chapter-4

Implementation , Testing and Maintenance

4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation :

The WhatsApp Chat Analyzer is implemented using **Python**, which provides powerful libraries for data processing and visualization.

Programming Language:

- Python

Libraries Used:

- Pandas – data preprocessing and manipulation
- NumPy – numerical operations
- Matplotlib – basic data visualization
- Seaborn – advanced visualization
- WordCloud – generating word clouds
- re (Regular Expressions) – parsing chat data

IDE/Development Tools:

- Jupyter Notebook / VS Code / PyCharm

Optional Tools :

- Streamlit – for building a simple web interface

These technologies enable efficient handling of chat data and generation of meaningful insights.

Tools/Technologies	purpose
python	Programming language
pandas	Data processing
matplotlib	visualization
seaborn	Advanced graphs
numpy	Numerical operations
streamlit	User interface

4.2 Testing Techniques and Test Plans :

Testing ensures that the system works correctly and produces accurate results.

Testing Techniques Used:

- **Unit Testing:**

Individual functions like preprocessing, analysis, and visualization are tested separately.

- **Integration Testing:**
Ensures all modules (input, preprocessing, analysis, output) work together properly.
- **System Testing:**
The complete system is tested using sample chat files.
- **Validation Testing:**
Checks whether the system correctly handles valid and invalid input files.

4.3 Installation Instructions :

To run the WhatsApp Chat Analyzer, follow these steps:

1. Install Python (version 3.x)
2. Install required libraries using pip:
pip install pandas numpy matplotlib seaborn wordcloud
3. Install Streamlit :
pip install streamlit
4. Download or copy the project code
5. Place the exported WhatsApp chat file in the project directory
6. Run the program:
 - For Python script: python main.py
 - For Streamlit app: streamlit run app.py

4.4 End User Instructions :

1. Export chat from WhatsApp in (.txt) format
2. Open the application
3. Upload/select the chat file
4. Click on the analyze button
5. View results such as:
 - Message statistics
 - User activity
 - Time-based graphs
 - Word frequency
 - Emoji usage
6. Interpret the visualizations displayed on the screen

Maintenance

The system requires minimal maintenance. However, the following actions may be performed:

- Updating libraries for better performance
- Modifying code to support new WhatsApp formats
- Improving visualization features
- Fixing bugs or errors if identified

Chapter-5

Results and Discussions

5.1 User Interface Representation :

The WhatsApp Chat Analyzer provides a simple and interactive user interface developed using **Streamlit**, which allows users to easily upload chat data and view analysis results without requiring technical knowledge.

The interface is designed to be clean, intuitive, and user-friendly, ensuring smooth interaction for all types of users.

Main Components of the User Interface

1. Title Section:

At the top of the interface, a clear heading such as “WhatsApp Chat Analyzer” is displayed to indicate the purpose of the application.

2. File Upload Option:

The interface provides a **file uploader** where users can upload the exported WhatsApp chat file in (.txt) format.

- Users can drag and drop or browse files
- Only supported formats are accepted

3. Sidebar Options :

A sidebar may be included to provide additional controls such as:

- Selecting a specific user for analysis
- Choosing type of analysis (overall / individual)

4. Analysis Button:

A button (e.g., “show Analysis”) allows users to trigger the processing of the uploaded chat data.

5. Display of Key Statistics:

Once the file is processed, the interface displays important metrics such as:

- Total number of messages
- Total words
- Media shared
- Links shared

These are usually shown using highlighted boxes or columns.

6. Visualizations Section:

The UI presents various graphs and charts, including:

- Monthly and daily activity graphs
- Most active users (bar chart)
- Word cloud
- Emoji analysis chart

These visual elements help users easily understand patterns in the data.

7. Output Area:

All results are displayed dynamically on the screen without requiring page reloads, making the interaction smooth and efficient.

5.2 Brief Description of Various Modules of the System :

The WhatsApp Chat Analyzer is divided into several modules, each responsible for a specific function. This modular approach makes the system organized, efficient, and easy to maintain.

1. Input Module

This module allows the user to upload the exported WhatsApp chat file in (.txt) format. It ensures that the file is correctly received by the system for further processing.

2. Preprocessing Module

This module cleans and prepares the raw chat data. It removes unnecessary system messages, extracts important fields such as date, time, user name, and message, and converts the data into a structured format.

3. Data Cleaning Module

This module handles missing values, removes invalid entries, and standardizes the data format to ensure consistency and accuracy in analysis.

4. Feature Extraction Module

This module extracts useful information from the processed data, such as total messages, word count, media messages, links, and emojis.

5. User Analysis Module

This module analyzes user activity by counting the number of messages sent by each user and identifying the most active participants in the chat.

6. Time Analysis Module

This module studies chat activity over time, including daily, monthly, and hourly trends, to identify peak usage periods.

7. Text Analysis Module

This module performs word frequency analysis by removing common stop words and identifying the most frequently used words in the chat.

8. Emoji Analysis Module

This module extracts and analyzes emojis used in the chat and determines their frequency and usage patterns.

9. Visualization Module

This module generates graphical representations such as bar charts, line graphs, and word clouds to present the analyzed data in an understandable format.

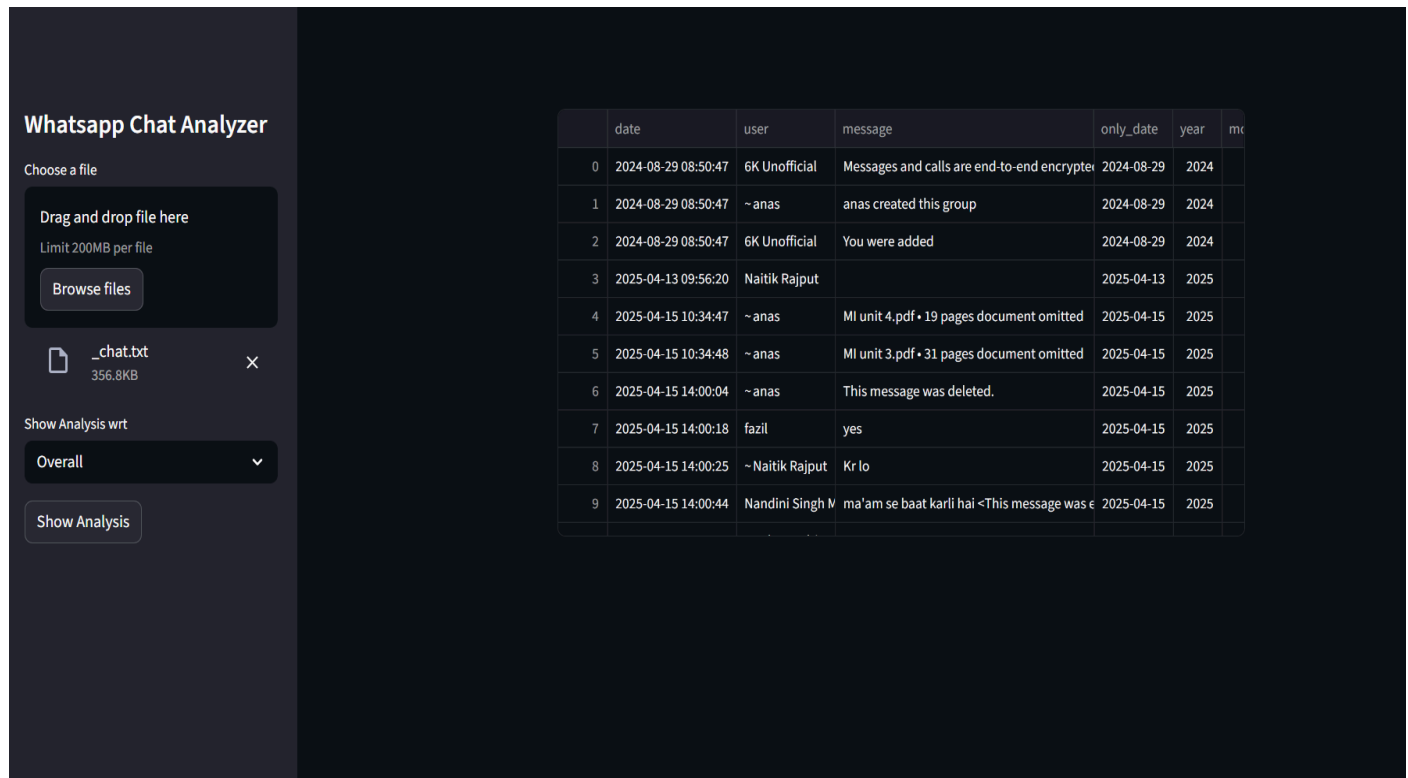
10. Output Module

This module displays the final results, including statistics and visualizations, to the user through the interface.

This modular structure ensures smooth data flow, better performance, and easier debugging of the WhatsApp Chat Analyzer system.

5.3 Snapshots of System with Brief Detail of Each :

5.3.1 Chat File Upload and DataFrame Creation

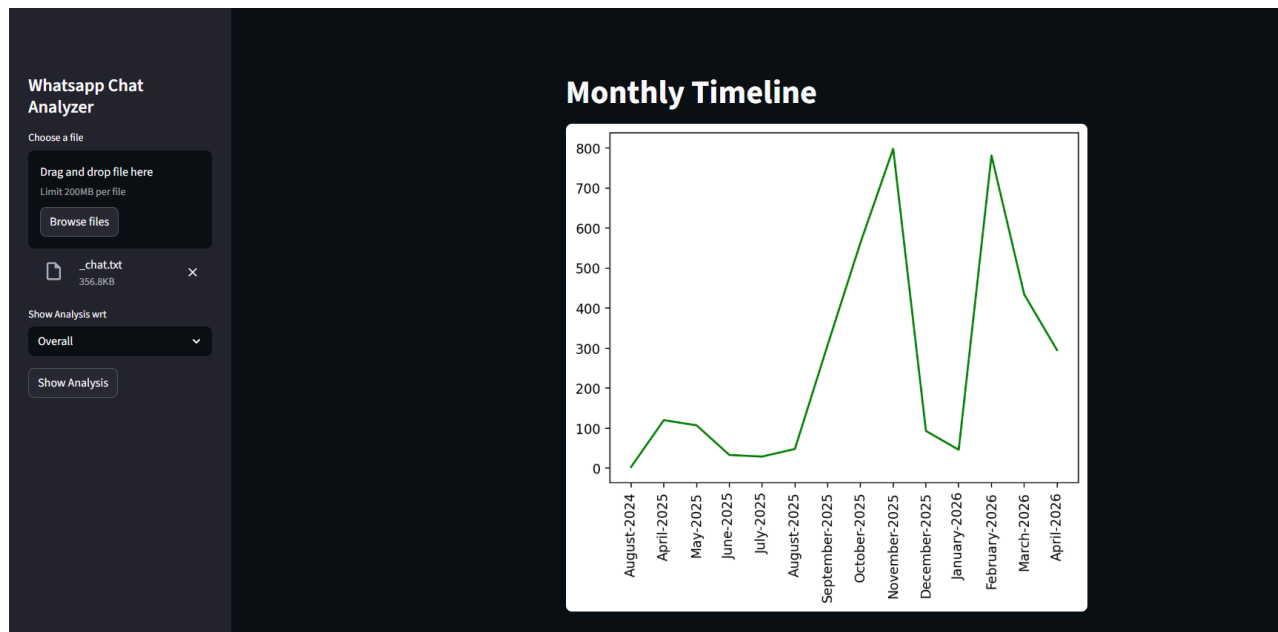


The screenshot displays the 'Whatsapp Chat Analyzer' interface. On the left, there is a sidebar with a file upload section titled 'Choose a file' containing a 'Drag and drop file here' area with a 'Limit 200MB per file' note and a 'Browse files' button. Below this, a file named '_chat.txt' (356.8KB) is shown with a close button. Further down, there is a 'Show Analysis wrt' dropdown menu set to 'Overall' and a 'Show Analysis' button. The main area on the right displays a DataFrame table with the following data:

	date	user	message	only_date	year	mo
0	2024-08-29 08:50:47	6K Unofficial	Messages and calls are end-to-end encrypted	2024-08-29	2024	
1	2024-08-29 08:50:47	~ anas	anas created this group	2024-08-29	2024	
2	2024-08-29 08:50:47	6K Unofficial	You were added	2024-08-29	2024	
3	2025-04-13 09:56:20	Naitik Rajput		2025-04-13	2025	
4	2025-04-15 10:34:47	~ anas	MI unit 4.pdf • 19 pages document omitted	2025-04-15	2025	
5	2025-04-15 10:34:48	~ anas	MI unit 3.pdf • 31 pages document omitted	2025-04-15	2025	
6	2025-04-15 14:00:04	~ anas	This message was deleted.	2025-04-15	2025	
7	2025-04-15 14:00:18	fazil	yes	2025-04-15	2025	
8	2025-04-15 14:00:25	~ Naitik Rajput	Kr lo	2025-04-15	2025	
9	2025-04-15 14:00:44	Nandini Singh M	ma'am se baat karli hai <This message was €	2025-04-15	2025	

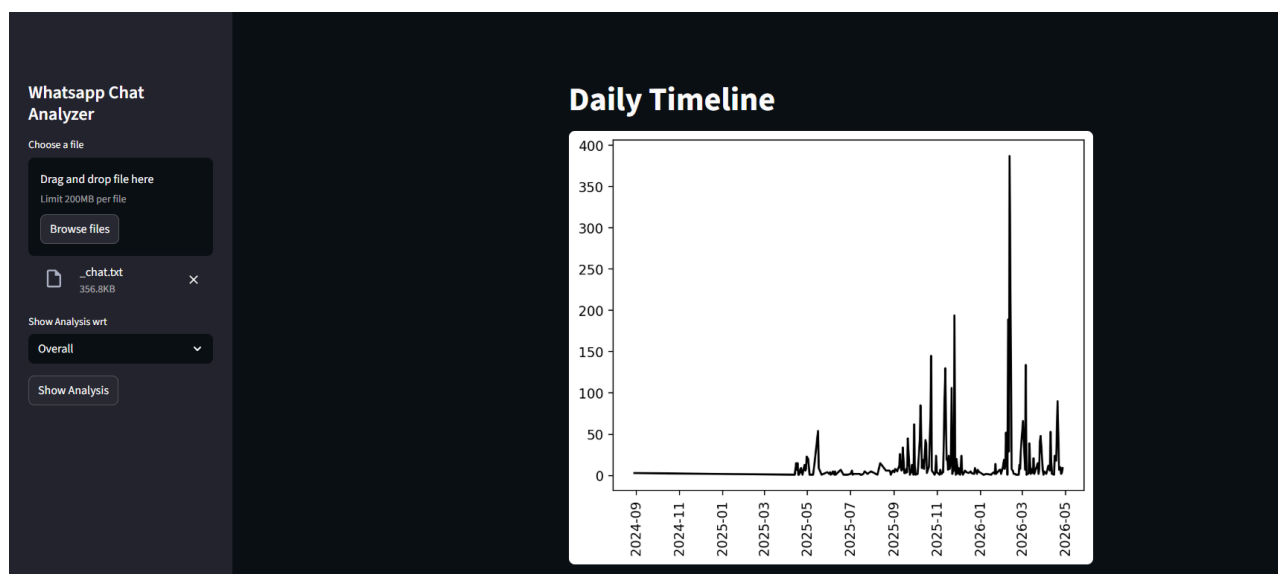
This snapshot shows the initial stage of the system where the user uploads the exported WhatsApp chat file in (.txt) format through the interface. Once the file is uploaded, the system reads the raw text data and processes it using preprocessing techniques. The data is then structured into a tabular format known as a DataFrame, where each row represents a message with attributes such as date, time, user name, and message content. This structured format enables efficient analysis in later stages.

5.3.2 Monthly Timeline Graph Visualization



This snapshot represents the monthly activity trend of the chat data. The graph displays the number of messages exchanged in each month over a period of time. It helps in identifying long-term communication patterns, such as which months had higher or lower activity. This visualization is useful for understanding overall engagement trends in the chat.

5.3.3 Daily Timeline Graph Visualization



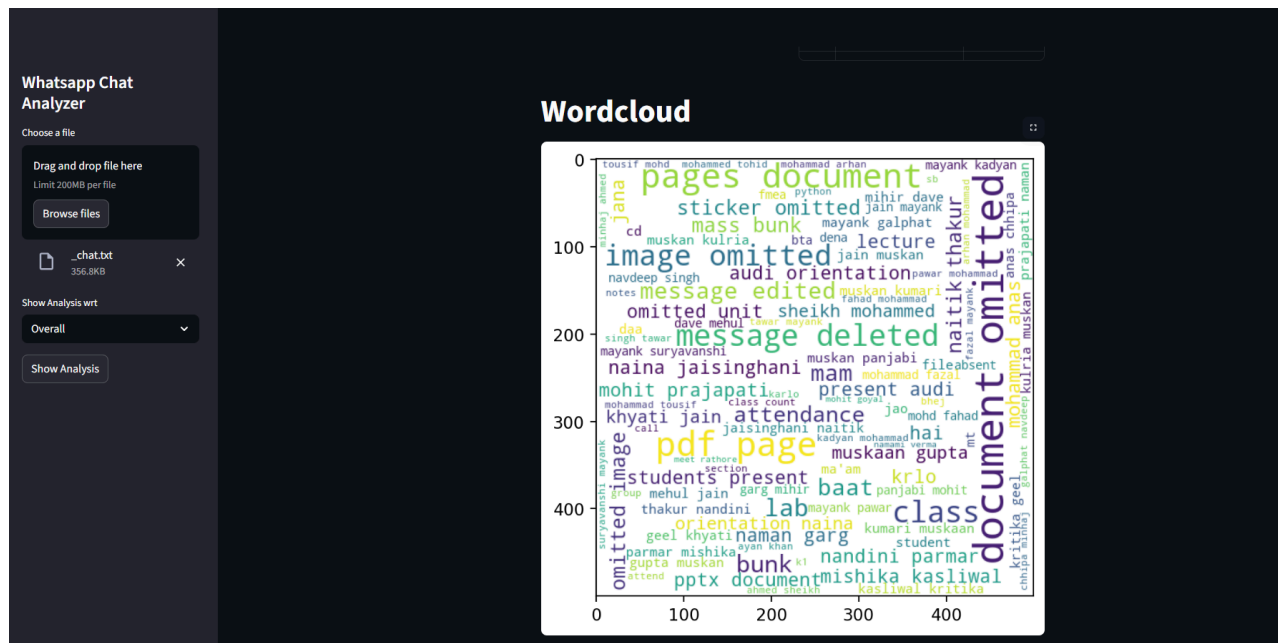
This snapshot shows the daily message activity in the form of a timeline graph. It represents the number of messages sent on each day. The graph helps in identifying specific days with high or low activity and provides a more detailed view compared to monthly analysis. It is useful for tracking short-term communication patterns.

5.3.4 Activity Map (Most Busy Days, Months, Weekly Activity)



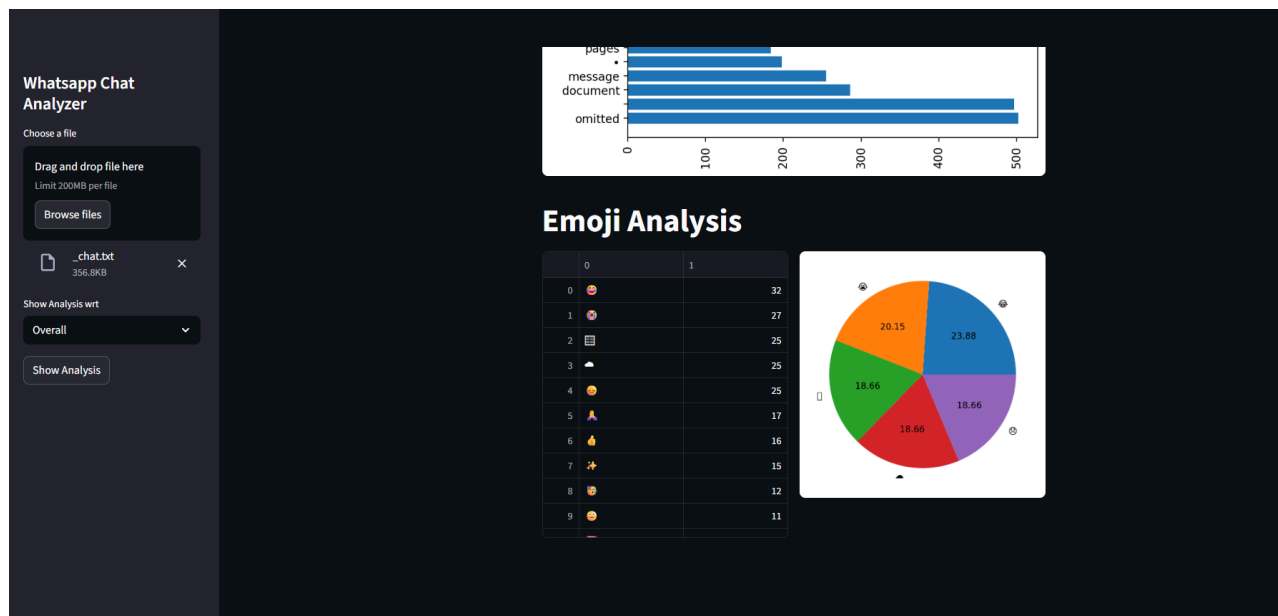
This snapshot displays the activity map of the chat, including the most active days, months, and weekly patterns. It highlights which day of the week (e.g., Monday, Sunday) has the highest message activity and which months are the busiest. The weekly activity map provides insights into user behavior and communication habits across different time periods.

5.3.5 Word Count Visualization



This snapshot illustrates the frequency of words used in the chat. It may be represented using a bar chart or word cloud. The visualization highlights the most commonly used words by users, helping in understanding the main topics or trends in the conversation. Common stop words are usually removed to focus on meaningful words.

5.3.6 Emoji Analysis



This snapshot shows the analysis of emojis used in the chat. It represents the frequency of different emojis, often in the form of a bar chart or list. This helps in understanding the emotional tone and expression patterns of users during conversations, as emojis are widely used to convey feelings and reactions.

Chapter 6 : Summary and Conclusion

The WhatsApp Chat Analyzer project was developed to analyze and extract meaningful insights from chat data using data analysis and visualization techniques. The system processes exported WhatsApp chat files in (.txt) format and transforms unstructured textual data into a structured format for analysis. Various functionalities such as message statistics, user activity analysis, time-based trends, word frequency, and emoji usage are implemented to understand communication patterns effectively.

The project follows a systematic approach, including data collection, preprocessing, feature extraction, analysis, and visualization. Tools such as Python, Pandas, Matplotlib, and Seaborn are used to perform efficient data handling and generate graphical representations. The use of a Streamlit-based interface makes the system interactive and user-friendly, allowing even non-technical users to analyze chat data easily.

The results obtained from the system provide valuable insights into user behavior, chat activity trends, and commonly used words and emojis. These insights help in better understanding communication dynamics in both personal and group conversations.

In conclusion, the WhatsApp Chat Analyzer successfully demonstrates how data analytics can be applied to real-world communication data. The project highlights the importance of transforming raw data into meaningful information through analysis and visualization. It also enhances practical knowledge of data preprocessing, visualization techniques, and handling real-world datasets.

Overall, the project meets its objectives and serves as a useful tool for analyzing chat data while also acting as a strong foundation for further enhancements and advanced analytical applications.

Chapter 7 : Future Scope

The WhatsApp Chat Analyzer project can be further enhanced in several ways to improve its functionality, usability, and analytical capabilities.

One possible improvement is the addition of a real-time chat analysis feature, where messages can be analyzed as they are received instead of relying only on exported chat files. This would make the system more dynamic and useful for continuous monitoring.

The project can also be extended by developing a more advanced and interactive user interface, including better visual dashboards, filters, and customization options for users to explore data more effectively.

Another enhancement could be the integration of multi-platform support, allowing analysis of chat data from other messaging platforms such as Telegram, Instagram, or Messenger, making the system more versatile.

The system can be improved to handle larger datasets more efficiently by optimizing performance and using advanced data processing techniques. This would make it suitable for analyzing very large group chats.

Additionally, features like exporting analysis reports in formats such as PDF or Excel can be added, enabling users to save and share insights easily.

Further improvements may include user comparison features, where multiple users' chat behavior can be compared visually, and more detailed analytics such as deeper text pattern recognition.

Finally, the project can be deployed as a web application or cloud-based system, allowing users to access it from anywhere without needing local setup.

In conclusion, the WhatsApp Chat Analyzer has strong potential for expansion, and with these enhancements, it can evolve into a more powerful and comprehensive communication analysis tool.

Appendix :

The appendix contains supporting material that is not included in the main body of the report but is useful for understanding the project implementation.

Appendix A: Sample Chat Data

A portion of the exported WhatsApp chat file used as input for analysis, showing the format of raw data.

Appendix B: Code Snippets

Important parts of the Python code used in the project, including:

- Data preprocessing functions
- Analysis functions
- Visualization code

Appendix C: Additional Outputs

Extra graphs, charts, or intermediate outputs that support the results but are not included in the main report.

Bibliography :

The following sources were referred to during the development of the WhatsApp Chat Analyzer project:

1. Official Python Documentation – <https://docs.python.org/>
2. Pandas Documentation – <https://pandas.pydata.org/docs/>
3. Matplotlib Documentation – <https://matplotlib.org/stable/contents.html>
4. Seaborn Documentation – <https://seaborn.pydata.org/>
5. NumPy Documentation – <https://numpy.org/doc/>
6. WordCloud Documentation – https://amueller.github.io/word_cloud/
7. Streamlit Documentation – <https://docs.streamlit.io/>
8. Stack Overflow – <https://stackoverflow.com/> (for debugging and problem-solving)
9. GeeksforGeeks – <https://www.geeksforgeeks.org/> (for reference and learning concepts)

